考核报告

——计算机科学与技术263班，刘宇浩

技术报告

基础入门部分

第一关：找到入口

本任务其实比较简单，我们先用 `ls` 找到key.txt，用 `cat` 查看，我们就知道了密钥是什么，也就是MOON，接着看一下 `exam --help` ，看到用法是

```
exam -e [密钥]
```

并且试着运行后会看出来exam会硬编码读取本目录下的input，所以我们要做的很简单：用mv把input.bin改名为input，运行后我们ls一下可以看到一个output，把它改名成flag即可通关

第二关：两段信号

这一关比较系统地考察了下对Linux一些常用指令的掌握，

和之前一样，我们要先用—help参数查看一下exam的用法

而我们知道，find可以用于递归查找文件，而grep可以搜索特定内容，于是一切都简单了：我们使用

```
find . -name “signal-A\*.\*”
find . -name“signal-B\*.\*”
```

就可以很顺利地找到这两个文件

找到这两个文件后，就该把它们拼接起来，这时候我们就该使用重定向符了，由于A要在前面，我们就执行

```
cat ./archive/2026/signal-A.bin \> input
cat ./inbox/signal-B.bin \>\> input
```

这样子input的准备工作就好了

接着我们要找到密钥，也就是含ACCESS_TOKEN的一个文件，这时候就需要grep和管道了，用find递归查找到所有文件，接着grep找到ACCESS_TOKEN,用管道连接起来，就是

```
find . -name "\*.\*" \| grep -r "ACCESS_TOKEN"
```

我们就能很简单知道，密钥是STAR

然后就简单了，直接运行exam，把生成的output直接改名flag即可结束。

第三关：读日志并修复输入

老规矩，进来先看一下exam的用法，可以发现还是一样要密钥，用ls总览一下后会发现一个key.txt，毫无疑问这个里面写的NOVA就是密钥，

接下来我们按照提供的日志路径去查看，很容易发现根本就没有日志：很显然，这需要我们先执行一遍exam后才有，自然，我们需要把这个input.bin改成input后再运行一下，通过日志看一下说什么。

日志里说的意思是，我们是19个字节，不能直接通过，字节数必须为4的整数倍，很显然我们需要一个空格，按常理而言这里要用sed，但我不会，所以我使用重定向插入一个空格

```
echo " " >> ./input
```

插入后就可以了，重新执行 `exam` 后把output改名flag就通过了

第四关：让命令找到路

对于不太懂shell的我来说，这题比较难，所以我只能使用AI生成shell，所以以下的都是我对照AI的代码进行的解读，可能有所缺陷：

依照shell的编写规则，先用一行

```
#!/bin/bash
```

声明一下这个是bash脚本。

在容器里直接 `echo $ROUTE_RESULT` 的话我们会发现啥也没有，所以为了稳妥，我们假设这个是可能存在可能不存在，所以先if一下，这玩意有就 `export` 一下，没有就 `export` 成 `"/workspace/flag"`

验证完 ROUTE_RESULT 后，我们就开始检验参数是不是两个，如果不是就 echo 一个 usage ，顺手把 ROUTE_RESULT 删掉一下，再退出64，

既然是两个参数，那下一步就该检验一下这两个参数是不是符合要求了，为了可读性好，我们定义 MODE 和 ITEM 这两个变量去存储这两个参数，

检验这两个参数需要用正则表达式筛选。

```
if [[ ! "$MODE" =~ ^(ok|retry|fatal)$ ]] || [[ ! "$ITEM" =~ ^[a-zA-Z0-9]{1,32}$ ]]
```

是不是符合要求，不符合要求一样， echo 一个 usage ，顺手把 ROUTE_RESULT 删掉一下，再退出64，

好不容易整完了这两个参数，我们接下来就把配置加载一下，用 source 去把 job.env 里的东西引入进来，将 ACCESS_TOKEN 和 TASK_TOOL_DIR 分别 export 到全局区，接着，我们还需要把整个工具目录加入 PATH 便于我们执行

```
export PATH="$TASK_TOOL_DIR:$PATH"
```

OK，那么接下来我们只需要调用这个工具，捕获输出，保存退出码即可，我们使用OUTPUT接收输出，RC接收退出码

有了退出码，我们接下来就是用case看什么码干什么，不过，由于我们需要一个会出，所以我们采用 printf 后面加 \n 的形式以实现严格的格式与换行控制。

第五关：照看后台任务

这一关难度比较高，也是AI生成的shell，以下也只是我的解读，

咱们先不管指定执行方式的第一行，直接看下面一行

```
WORK_DIR="${WORK_DIR:-/opt/recruitment}"
```

这一行的意思是如果有 WORK_DIR 则不做改变，没有则指定为 /opt/recruitment

接着，我们把 PID_FILE 和 WORKER 定义一下，分别就是存储PID的文件路径和slow_worker的路径

接下来，我们定义一个函数 is_running ， is_running 先判断PID_FILE是否指向一个存在的文件，如果存在，就定义一个变量 pid ，并把从 PID_FILE 中读取出的数据存入 pid 变量内，拿到了这个pid，就先检验这个pid是否存在了，我们用 kill -0 发送信号来检验，如果这个进程在，我们就再用 ps 搭配管道和 grep 查查这个pid是不是我们的slow_worker，如果是就可以返回0了，表示在运行，以及，以上如果任何一个条件不满足就返回1，也就是不在运行

然后我们来写启动函数 do_start 函数，题目要求不能重复运行，那我们就调用 is_running 来看看，在运行就直接返回0结束，不在运行的话，就先创建这个目录，用 -p 进行递归创建，一行代码就能防止由于目录不存在而报错。接下来，我们使用 nohup 创建进程， > /dev/null 抛弃标准输出让它不在屏幕上打印任何内容， 2>&1 就是把错误输出也输入和标准输出一起的地方，翻译：全扔了，最后一个 & 使程序在后台运行，

那么就开始写 status 函数，很显然这个是最好写的，我们只需要调用 is_running ，看 is_running 的返回值行事就可以了。

接下来是 do_report 函数，我们依然先调用 is_running 函数，在运行就返回1，没有运行，那就拿出PID，然后我们再定义一个 report_file ，也就是resource.txt，接着，我们把PID用 echo 和重定向把这个放入 report_file ，接着我们用 ps -p 加PID截取ps快照并存入 report_file ，同样，
用 top -b -n 1 -p PID ，批处理模式处理top一次并使用 pid 变量指向的 pid ，也一样用重定向追加到 report_file 的末尾。就可以 return 0 了。

然后我们写 do_stop 函数，我们依然先调用 is_running 函数，在运行就返回0，不在运行就取PID，拿到PID后，我们用 kill 向这个进程发送TERM信号，也就是15号信号，让程序正常终止，依旧扔掉错误输出，然后，等一秒，等晚会看看进程还在不在，不在就在补上一个TERM信号，然后再睡1秒，反正这个程序没了后就把 PID_FILE 删了，防止混淆，最后直接返回0.

以上4个函数全部写完以后，剩下的就十分简单了，只需要根据一个 case 判断输入参数并分别执行对应的函数，其他的就是输出usage并返回64，就结束了。

C/CPP进阶方向

读数统计

本任务先判断 n 是否大于10000，如果是则程序返回1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

接着我们把需要的四个变量 count , sum , min , max 定义出来并初始化一下，考虑到后面我们是通过比大小动态更新 min 和 max ，所以，我们把 min 初始化为 1000 ， max 初始化为 -1000 。接着就可以开始循环读取了

循环的控制我们通过 count 变量和 scanf 是否成功读取来控制，只要 scanf 还能成功读取且 count 没有达到 n ，就继续循环

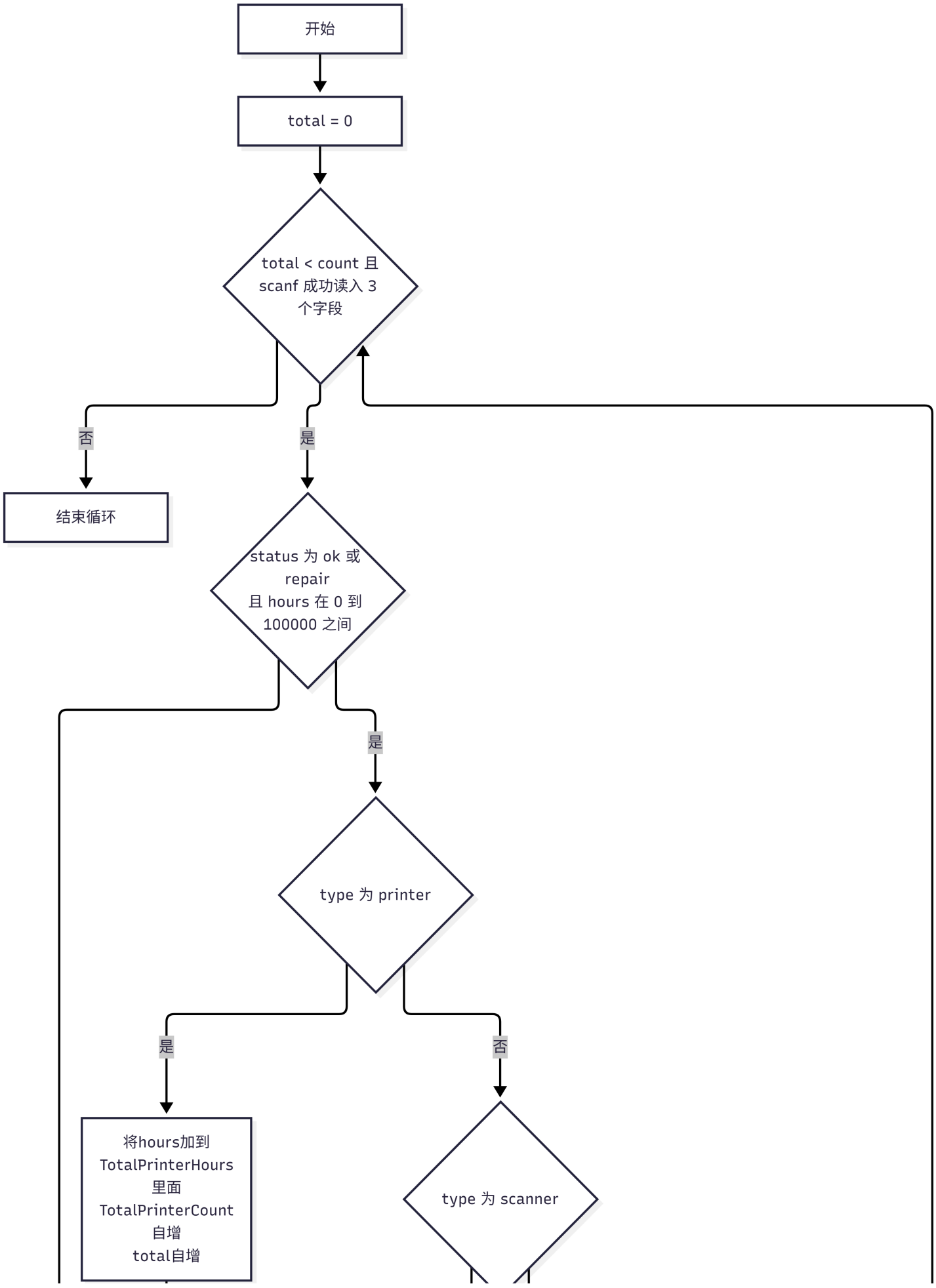
我们将读取的数据存储在一个 temp 变量中， count 变量表示实际读取的数目，每次读取一个后 count 就加一，并且分别将这个数与 min 和 max 对比，如果 temp>max ,就令 max=temp 。而如果 temp<min ，则令 min=temp ，同时把这个是数加入 sum

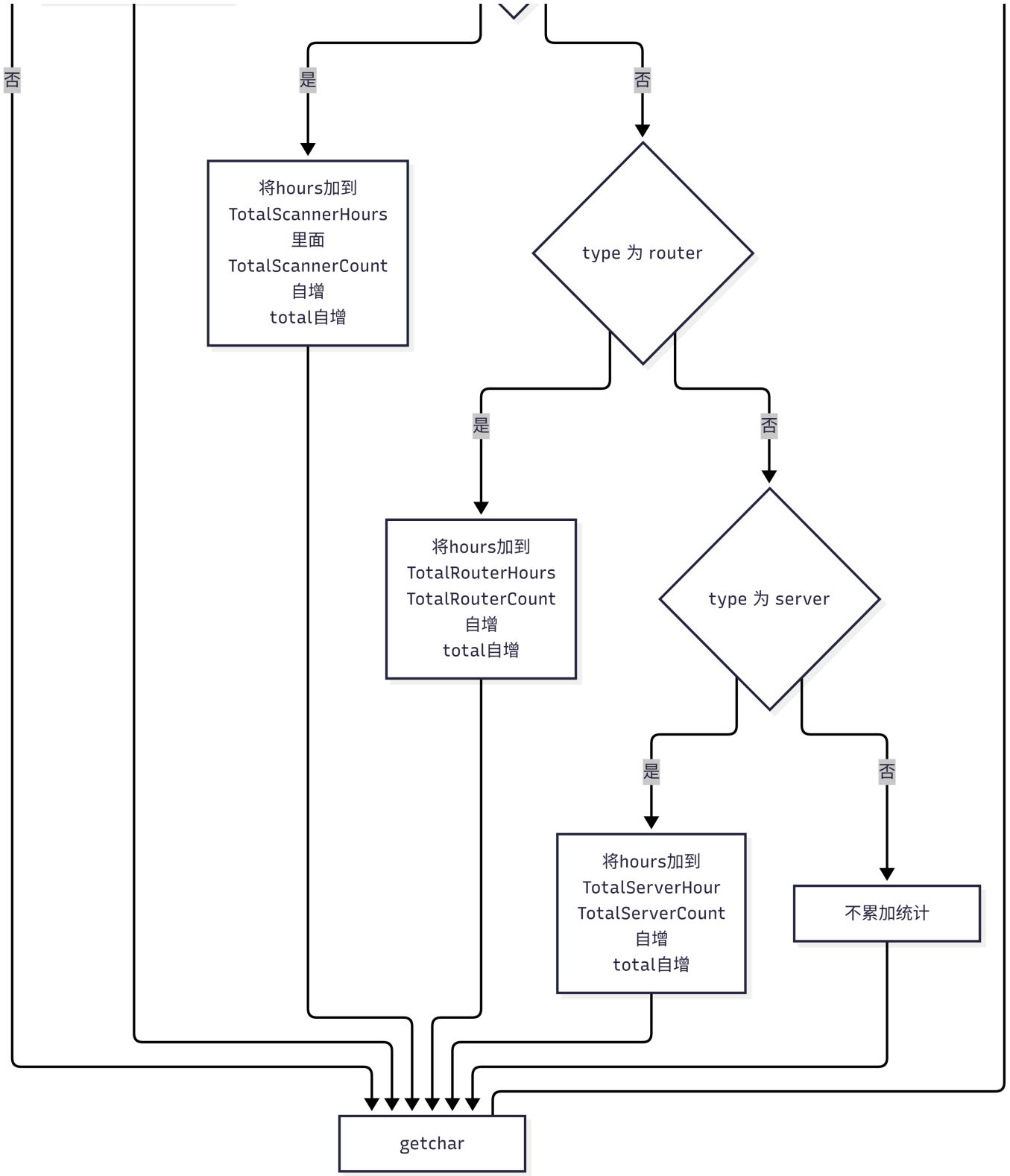
循环结束后一切就简单了，我们只需要根据格式，把 `count` ， `sum` ， `min` ， `max` 分别输出就可以了。

设备巡检记录

本任务先消除掉读取count后残留的换行符，避免后续scanf无法读取，接着定义八个变量，分别为printer，scanner，router，server这四个分别对应的总时长与次数，接着进入循环。

在循环里，我们先检验一下scanf是不是读到了这三个数据，要是读不到就脱离循环，同时我们使用total变量控制循环次数，如果在scanf还能读取的情况下total还没达到输入的count，那么就持续循环读取。（以下为循环流程图）


开始
total = 0
total < count 且 scanf 成功读入 3 个字段
否
结束循环
是
status 为 ok 或 repair 且 hours 在 0 到 100000 之间
是
type 为 printer
是
将hours加到 TotalPrinterHours 里面
TotalPrinterCount 自增
total自增
否
type 为 scanner



读取到数据后，我们先对照一下status是不是ok或repair，以及time是不是在

$$0, 100000$$

内，如果不是total不自增，跳入下一循环，是的话继续看前面type是啥，并把time加入对应的总时长，次数加1，total加1，没有继续下一循环，每次进入下一个循环都先用getchar清除一下换行符。

读取完所有的数据就该判断busiest是哪个了，我们先假定busiest是1(不初始化为0的原因是为了防止后面全都匹配不到导致无busiest，当然，我们还有另一种办法，将TempTime初始化为-1就可以了)，并定义一个TempTime变量初始化为0，用于比较，我们首先对比一下Temptime和printer的时间TotalPrinterHours，如果前者更多就保持busiest和TempTime不变，如果后者更多就把TempTime和busiest和后面那个靠齐，以此推下去将4个按顺序对比，就可以得到时长最多且最靠前的。

固定窗口读数工具

本任务的处理十分简单，对i进行取CAPCITY的余（我个人认为，这里的8不能写死，定义这个CAPCITY是为了实现解耦，便于根据需求修改所需窗口大小），使得当i达到CAPCITY后可以回绕从下标0存储，覆盖掉最早的一个。

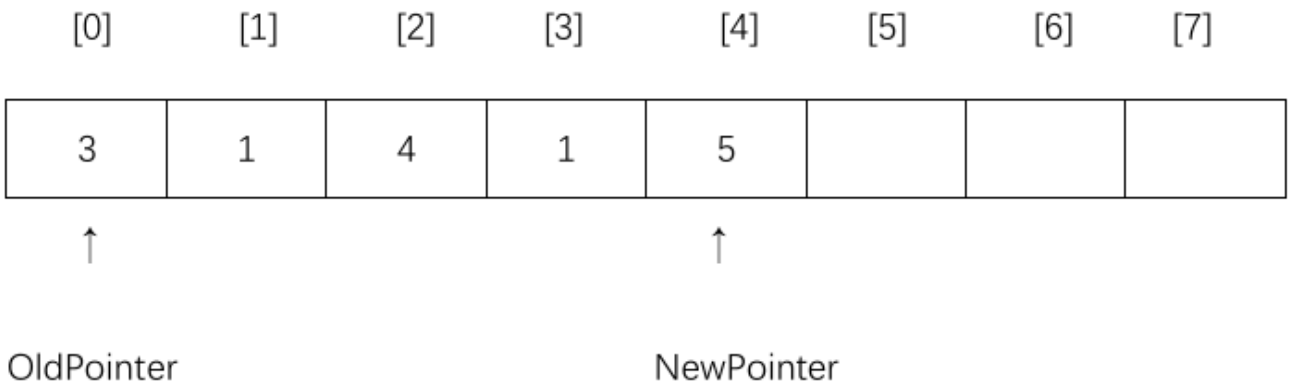
至于keep的控制，很简单，我们设计一个if，当keep没达到8就自增，达到了8就不自增

同时，我们定义一个NewPointer和OldPoint分别用于指向最新的和最旧的数据，在i未超过8时，keep正常增加，而当keep达到8后，keep不再增加，同时OldPointer指向对i+1取8的余数，这样OldPointer始终

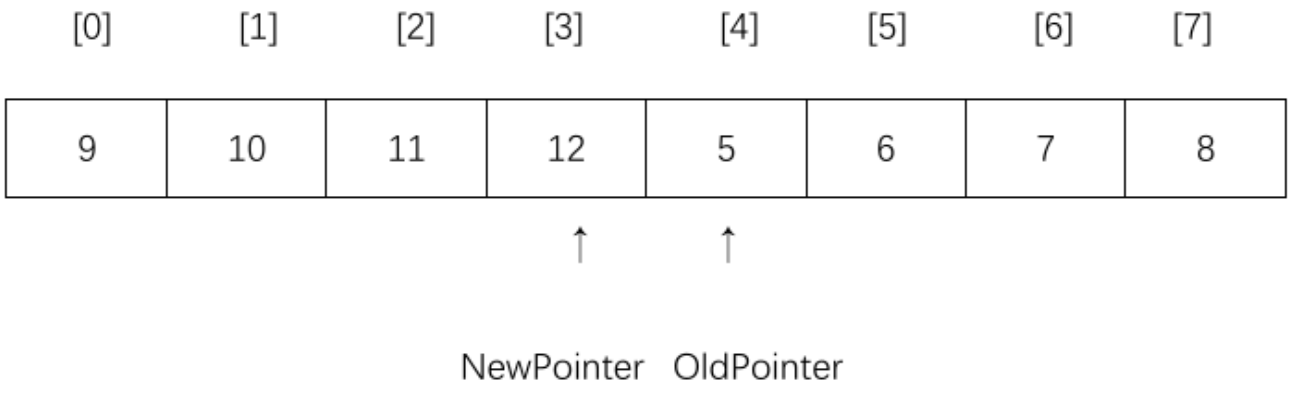
指向NewPointer前面的一个，最快被覆盖的就是最早的那个

以题目里的样例为例

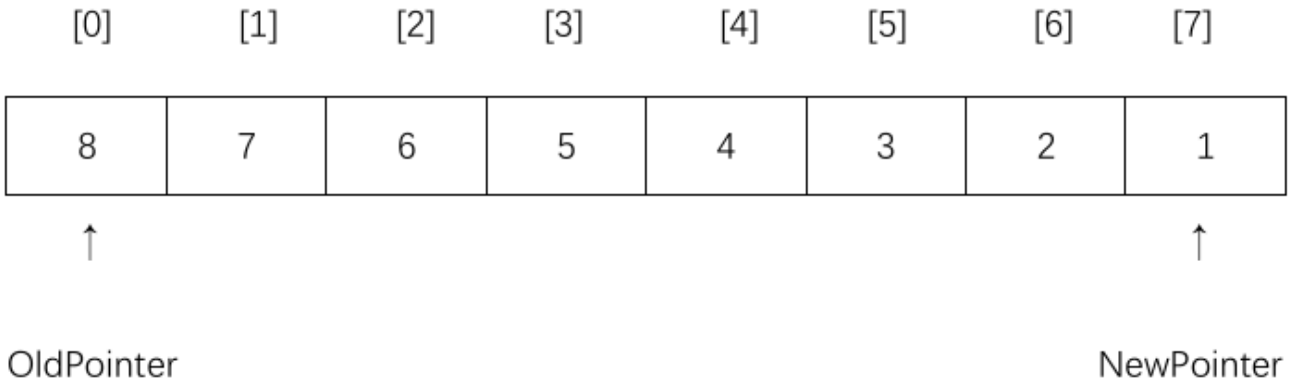
样例 1：(3，1，4，1，5)



样例 2：(1，2，3，4，5，6，7，8，9，10，11，12)



样例 3：(8，7，6，5，4，3，2，1)



多文件统计工具

这个任务要求修复由多个文件组成的文件统计工具，在main()里，使用的格式是：

void print_report(int count, long long total, int smallest, int largest)

而在format.c和format.c中，参数顺序颠倒，我们要做的比较简单，就是对照main()里的调用类型，反过来修改format.h和format.c里参数列表里的类型，使之匹配

除了这个，我们还需要修改CMakeLists.txt，给add_executable里面加上format.c，并且加上这两句：

set(CMAKE_C_STANDARD 11)

set(CMAKE_C_STANDARD_REQUIRED ON)

第一行指定使用C11标准，第二行执行强制要求使用这个标准。

加上这两行后，整个任务就完成了。

自然，不要忘记把文件从c文件夹里拿出来。

感悟报告

基础方向

找到入口

这一关是所有考核关最简单的一关，但必须说实话的是，看到这一关我是懵的：我的想法是“这些字怎么会串成一句话呢？”，虽然现在回头过来看其实写得很简单，但必须说，这个新手指导做得有点差了。

两段信号

这一关也其实比较简单，但，说真的，国庆七天我一直在学git，而Linux没有着重学习，我们寝室里都是这样，因为我们之前问AI说要重点学哪里，AI说重点学git，Linux不会考多少，现实却是我在那一遍遍地翻着Linux教程。对照着女娲补天.....

读日志并修复输入

经受了前面两关的“拷打”，在第三关我终于适应了起来，开始能看懂题目在讲什么了，不过最开始写时，我第一想法是用nano去加入一个空格，但随后日志就告诉我 `input` 变成了21个字符，我明白估计是多了啥东西，后来求助AI，AI让我用重定向去输入空格，然后就过关了

让命令找到路

本来前面过了第三关我觉得这基础也不咋地，随后看到要shell脚本时，心脏骤停————根本就没学过shell的一点知识，于是只能去找shell的教程，要什么现场看，折腾了半天（真的是半天，12个小时）都没能搞定，一直检查不通过，最后无奈只能全部用AI

要知道在此之前我基本没有用AI去生成过代码，这里两个shell几乎就是我为数不多纯AI的代码，我真的对shell懂得不多，虽说结合着教程我也是看懂了这个shell脚本在干什么。

照看后台任务

最折磨的任务，没有之一，前面的是让我感到快疯了，这个是真的差一点就疯掉了：AI生成了好几遍都没有通过，别的问题我基本都只会开一轮对话，唯独这个的超时的报错，我为了解决这个开了足足6轮对话，反复修改提示词，不止我，我们全寝都开着各自最常用的AI在分析这个shell脚本怎么写。最后好不容易成功我整个人都差点虚了，看代码也难得多，啃下来更是没少花多少时间升天

C/CPP进阶方向

读数统计

在写这读数统计时，我最开始采取的方案是采用 `fgets` 读取一整行，再通过 `sscanf` 依次读取数据，由于 `sccanf` 无法自动移动，需要一个巨大的字符数组去接收，而且需要防止缓存区溢出，而且会因未知原因造成输出与预期不符合：

```
char buffer[40000];

getchar();

fgets(buffer,sizeof(buffer),stdin);

//printf("%s",buffer);
//什么你问这行注释是什么意思？这当然是我在自己Linux虚拟机上试验fgets函数效果用的测试语句

int max = -1000,min = 1000,count = 0,sum=0;

int temp = 0, offset = 0 ,size = 0;

while (count < n && sscanf(buffer+offset,"%d%n",&temp, &size) == 1)

{


//由于sscanf无法自己移动所以需要我们用一个size来表示读取了多少个字符，并将其加到offset上

    offset += size;

    count++;

    sum += temp;

    if (temp\>max)

    {

        max = temp;

    }

    if (temp\<min)

    {

        min = temp;

    }
```

```
}
```

所以在最后和AI讨论后改为采取 `scanf` 读取，更加简单（当然，前者一般在实际业务使用更多，但对我而言难度比较高，所以先采用 `scanf` 方案）

设备巡检记录

这一关也是问题比较多，原本我以为用 `scanf` 只要读取成功就一定返回1，但实际情况是输入足够数量的数据没有结束，经过询问AI，得知 `scanf` 的返回值取决于成功读取了几个数据，所以在此处，scanf的返回值应该为3。

还有，不知道是不是是太久没摸C语言了，我闹了两个笑话：一个是在 `switch` 里使用字符串，但其实 `switch` 里面只能用 `int` ，另一个是在C语言里，字符串的对比不能用 `==` ，要用 `strcmp` 函数，当然，或许也可能是我以前做的程序很少出现和字符串打交道的情况，以前写 `switch case` 时，经常用枚举来方便自己阅读

除了这个，在 `busiest` 上，前面技术报告里我也说了，本来是将 `busiest` 初始化为0的，根据对比时间来确定busiest是哪一个(要换我以前，估计又要开始写枚举了)，当然，由于查出了bug，AI建议我把代码改成 `busiest` 初始化为1，但事后我又想到另一种方法，也就是把 `TempTime` 初始化为-1，这样也可以防止这种全0的情况，不过由于已经提交了也不太好改了。

固定窗口读数工具

我敢说这是整个C/C++方向对我最友好的一关：除了最开始的回绕不明白什么意思，不过用AI查了一下就知道了这个是什么意思，一开始我想到的是队列，但是我们又不可能从头写一个队列：太费时间了，所以，我采用两个指针(说白了就是两个记录下标的变量，不是那种记录变量地址的指针变量)，一个指向最早的，也就是 `OldPointer` ，一个指向最新的 `NewPointer` 。

不过在写关于这两个变量怎么动的时候，我又面临一个问题：在输入数据未达到8时， `OldPointer` 不需要移动，而到8以后我们就要移动 `OldPointer` ，这个我后来想到了 `keep` ，利用 `keep` 是否小于8的两种情况来分别处理 `OldPointer` 的不同移动方法，

原本关于 `OldPointer` 的移动方法我也十分头疼，后来我想到，最早进来的也会最先被覆盖，那么哪个最快被覆盖哪个就是最先进来的，也就是i+1的取余。所以很快就完成了，by the way，这估计是最快完成的一关。

多文件统计工具

关于多文件编程，虽然以前我自己做程序时，也常常把各功能块分离，便于自己查找，不过由于以前比较依赖IDE，所以关于怎么把这几个文件放一起用的CMake我不太了解，自然又去补了一番课，在弄这

个CMakeLists.txt时，我也闹了不少笑话，照抄hints.txt时，忘记把_CXX_改成_C_了，当然，由于我不懂逆向，所以我也不知道CMake是以一个什么样的心情看着这个前面说语言是C语言后面又说要CXX 11的标准的文件的(狗头保命)。

由于那天做这个时我前面在晚自习上刚完成固定窗口读数工具，所以我一股子劲地直接来对付这一关，所以甚至搞出了“把 `largest` 的类型改成 `long long` ”的闹剧，好在最后是修复好了，也发现了这个类型的不匹配，其实主要还是心焦了，直接看着 `format.h` 时就开始直接修改了，后来仔细检查 `main` 调用 `print_report` 时才注意到这个问题，从头调整了一遍，至于过没过关……我在这都排了两天队了都，我还是不知道我现在是不是确实没有问题了，当然，现在是已经恢复了，不过我提交后还是说无法调用，我还纳闷这还有什么问题？转头一想，是不是因为这些没在 `/workspace` 目录下？于是我干脆直接移动过来并直接提交，才终于通过了。